

Final Project - Data Science in Cyber Phishing Detection

Critical Evaluation and Reproduction of a Machine-Learning Tutorial

Selected source: [shreyagopal/Phishing-Website-Detection-by-Machine-Learning-Techniques](#)

Dataset: 5.urldata.csv - 10,000 URL records and 17 extracted features

Submission package: PDF report, executable notebook, README, data, figures, and outputs

Executive Summary

This project evaluates a public phishing-detection machine-learning repository. The selected source defines phishing website detection as a binary classification problem, extracts URL/domain/HTML/JavaScript indicators from phishing and legitimate URLs, trains supervised models, and reports XGBoost as its best-performing model. I reproduced the modeling task using the provided final dataset and added a stricter critical evaluation inspired by the course lectures: EDA, missing and special values, duplicate analysis, correlation and association, feature engineering, train/test methodology, model comparison, evaluation metrics, false-positive/false-negative analysis, and explainability.

The dataset contains 10,000 rows and 16 original numeric features plus a domain string and label. It has no missing values and an exactly balanced distribution of 5,000 legitimate and 5,000 phishing records. This balance is useful for learning but unrealistic for deployment. A major finding is that the dataset contains 5,626 duplicated full rows and 6,447 duplicated domain values, which can make random-split evaluation overly optimistic.

On a random stratified 80/20 split, the best model in my reproduction was Random Forest with accuracy 0.959, precision 0.993, recall 0.925, F1 0.958, MCC 0.920, and ROC-AUC 0.995. On the stricter grouped split by domain, the best model was Gradient Boosting with F1 0.875 and ROC-AUC 0.939. The performance drop supports the source at a high level - the extracted features are predictive - but weakens any claim that random-split performance is production-ready evidence.

My conclusion is that the selected source is useful as an educational tutorial and baseline, but its evaluation is incomplete as a cybersecurity study. Stronger evidence would require temporal validation, realistic class prevalence, leakage checks, threshold selection based on operational costs, and testing on newer unseen phishing campaigns.

1. Summary of the Source

The selected source is the GitHub repository [shreyagopal/Phishing-Website-Detection-by-Machine-Learning-Techniques](#). It satisfies the project requirements because it defines a concrete cybersecurity problem, proposes a machine-learning solution, includes implementation notebooks, and provides a final dataset that can be reproduced without crawling live websites.

Source overview

Property	Value
Problem	Classify a URL/domain as phishing (1) or legitimate (0).
Importance	Phishing is a social-engineering attack that can lead to credential theft, fraud, malware installation, and organizational compromise.
Proposed solution	Extract URL, domain, HTML, and JavaScript features and train supervised classifiers.
Dataset	10,000 records: 5,000 phishing and 5,000 legitimate URLs.
Original features	17 extracted indicators, including URL length/depth, IP in URL, @ sign, TinyURL, prefix/suffix, DNS record, web traffic, domain age, iframe, mouse over, right click, and redirects.
Original models	Decision Tree, Random Forest, MLP, XGBoost, Autoencoder, and SVM.
Original main claim	The repository README reports XGBoost as the best model with 86.4% model performance.

The source states that phishing URLs were sampled from PhishTank and legitimate URLs from the University of New Brunswick URL dataset. The final CSV in the repository is therefore a processed feature table, not raw web pages.

2. Critical Evaluation of the Author Claims

The main claim is that supervised machine learning can detect phishing websites from extracted URL and webpage features, and that XGBoost achieved the best performance among the author's models. My

reproduction supports the general direction of this claim: the features clearly contain predictive signal. However, the original evaluation methodology leaves important questions unanswered.

- Evidence support: The dataset is separable enough for tree-based models to achieve high random-split performance. This supports the idea that handcrafted URL and webpage features are meaningful.
- Methodology limitation: The original evaluation appears to rely mainly on a random 80/20 split. Because this dataset has many duplicated rows and duplicated domains, random-split metrics can overstate generalization.
- Temporal limitation: No timestamp column is provided. The course lecture on time series and pipelines emphasizes that train data should precede test data when time matters. In phishing, time matters because campaigns, domains, and evasion strategies change.
- Prevalence limitation: The dataset is artificially balanced. In real systems, benign URLs are far more common, so precision and false-positive rate can become more important than accuracy.
- Feature-generation limitation: The source provides a final feature table, but the full data-collection process is hard to audit because live phishing pages disappear and external services change.
- Conclusion validity: The conclusion is justified as a tutorial baseline, but not as proof of a production-ready phishing detector.

The strongest contradiction to a naive interpretation of the original results is the gap between random-split and grouped-split performance. A model that performs extremely well when duplicated or related domains can appear in both train and test sets may perform worse on genuinely new domains. This is exactly the type of methodological issue the project instructions ask students to identify.

3. Feature Engineering Analysis

The original source already performed substantial feature extraction. The features are meaningful from a cybersecurity perspective because phishing URLs often use suspicious lexical structure, shortened URLs, young or low-reputation domains, redirection, and webpage tricks such as iframe abuse or disabled right-click behavior.

Top Spearman correlations with the label

Model/Item	abs_spearman
URL_Length	0.5413
Prefix_Suffix	0.3027
URL_Depth	0.1501
Have_At	0.1184
Web_Traffic	0.1088
iFrame	0.0984
Domain_Age	0.0851
Have_IP	0.0744

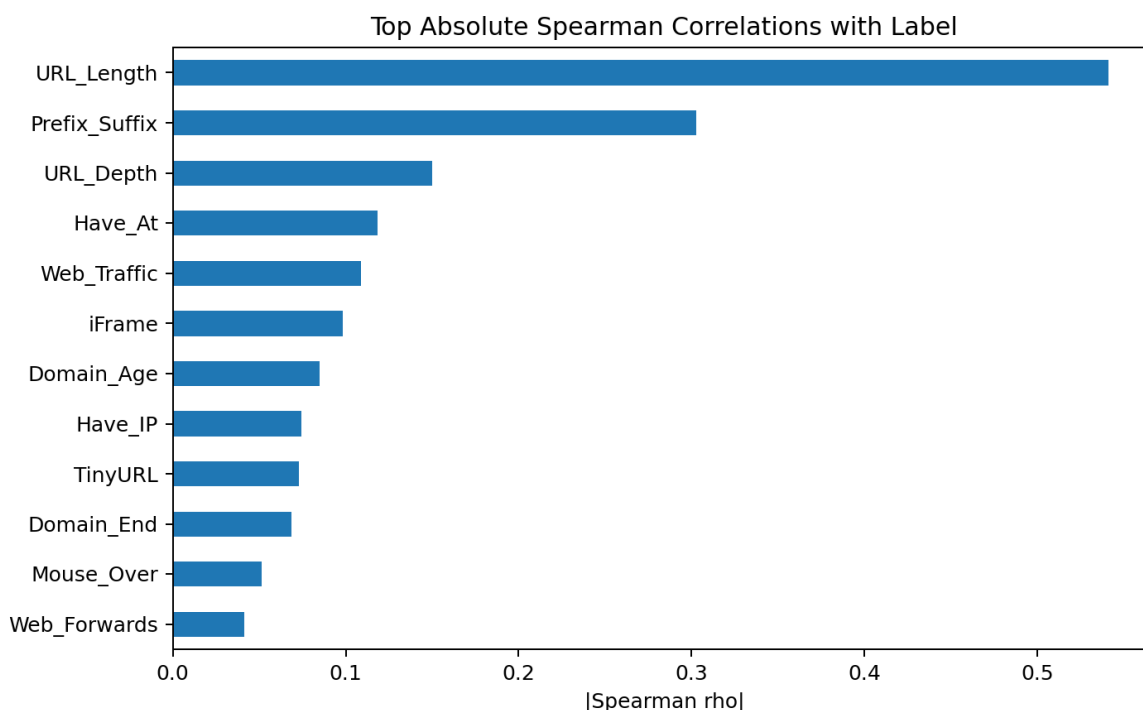


Figure 1. Absolute Spearman correlations with the phishing label.

I used Spearman correlation rather than Pearson because the feature table is mostly binary or ordinal. Spearman captures monotonic association and is more robust for this type of data. Redundancy was checked by looking for feature pairs with absolute Spearman correlation at least 0.70. The most notable redundant pairs were iFrame with Mouse_Over and Mouse_Over with Web_Forwards. Such redundancy can be handled by feature selection, regularization, permutation importance, or by removing one feature from highly correlated groups.

My additional feature engineering used only safe structural information from the Domain string: domain length, digit count, hyphen count, dot count, subdomain count, whether the domain starts with www, TLD length, and whether the TLD is common. I intentionally did not one-hot encode the raw domain. One-hot encoding thousands of domains would likely memorize the dataset and harm generalization to new phishing campaigns.

Top feature importances - best random-split model

Model/Item	importance
domain_length	0.3193
URL_Length	0.2539
URL_Depth	0.1005
domain_dot_count	0.0622
subdomain_count	0.0523
Prefix_Suffix	0.0276
tld_length	0.0272
domain_hyphen_count	0.0243
Web_Traffic	0.0180
domain_digit_count	0.0164

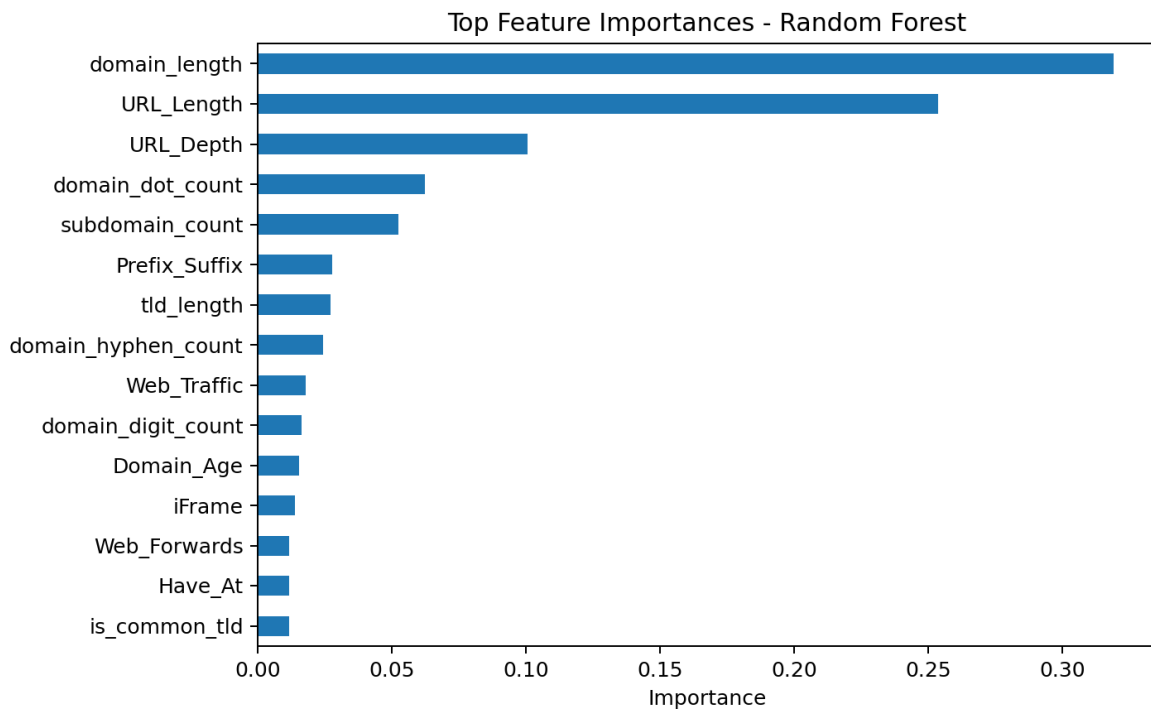


Figure 2. Top feature importances for the best random-split model.

4. Reproducibility Analysis

At the dataset and modeling level, the work is reproducible: the repository includes the CSV file, model-training notebook, feature-extraction notebook, and code artifacts. I was able to load the provided CSV and train models successfully.

- Available files: The final CSV and notebooks are available in the repository.
- Dependencies: Standard Python data-science libraries are sufficient for my reproduction: pandas, numpy, matplotlib, and scikit-learn.
- Hidden preprocessing: The main hidden risk is not within the CSV loading code, but in the earlier data-collection and feature-extraction process. Phishing pages are volatile, so rerunning collection later may not reproduce the same dataset.
- Temporal reproducibility: The dataset lacks collection timestamps; therefore temporal splits and drift analysis are not possible from the provided file.
- Overall reproducibility: Good for the final feature table; moderate to weak for the full real-world data-generation pipeline.

5. Experimental Results

I performed data inspection, missing-value analysis, class-balance analysis, crosstabs, Spearman correlation, redundancy analysis, feature engineering, model training, random-split evaluation, grouped-split evaluation, cross-validation, feature importance, threshold analysis, and false-positive/false-negative error analysis.

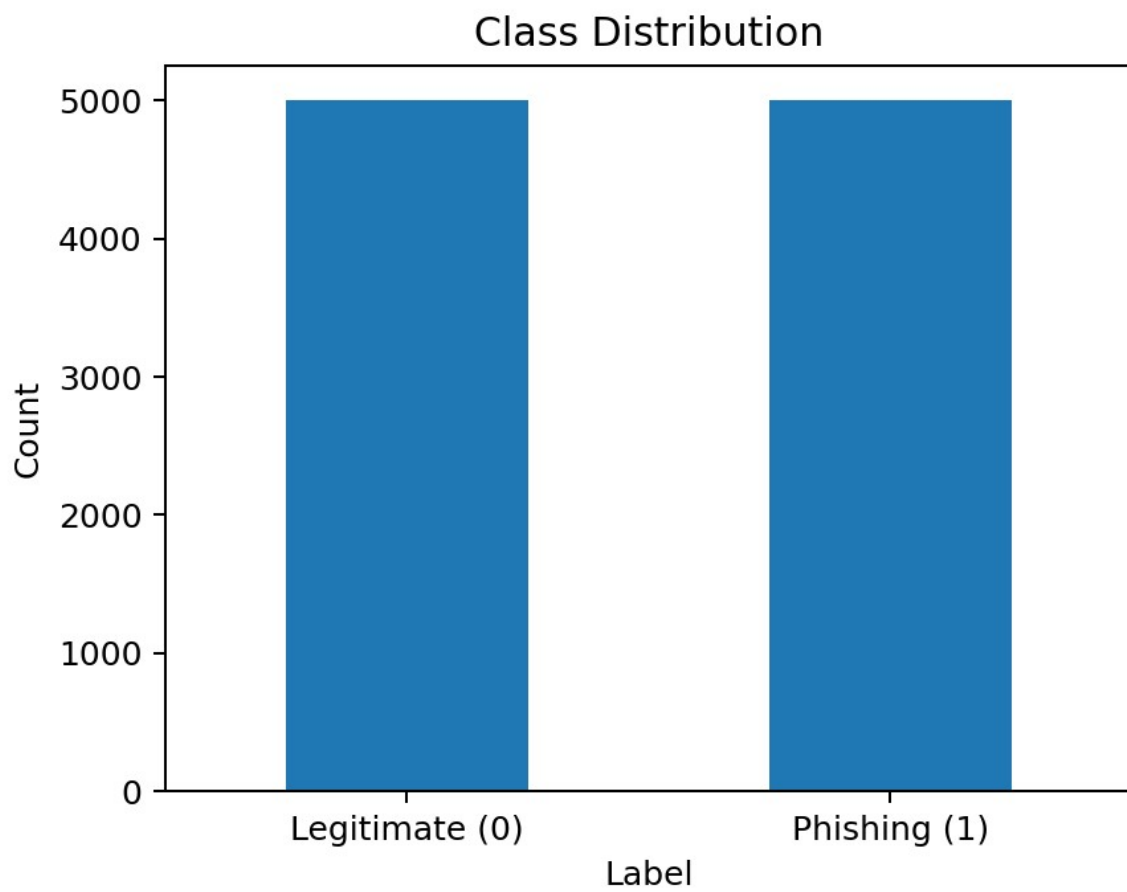


Figure 3. Class distribution: the dataset is exactly balanced.

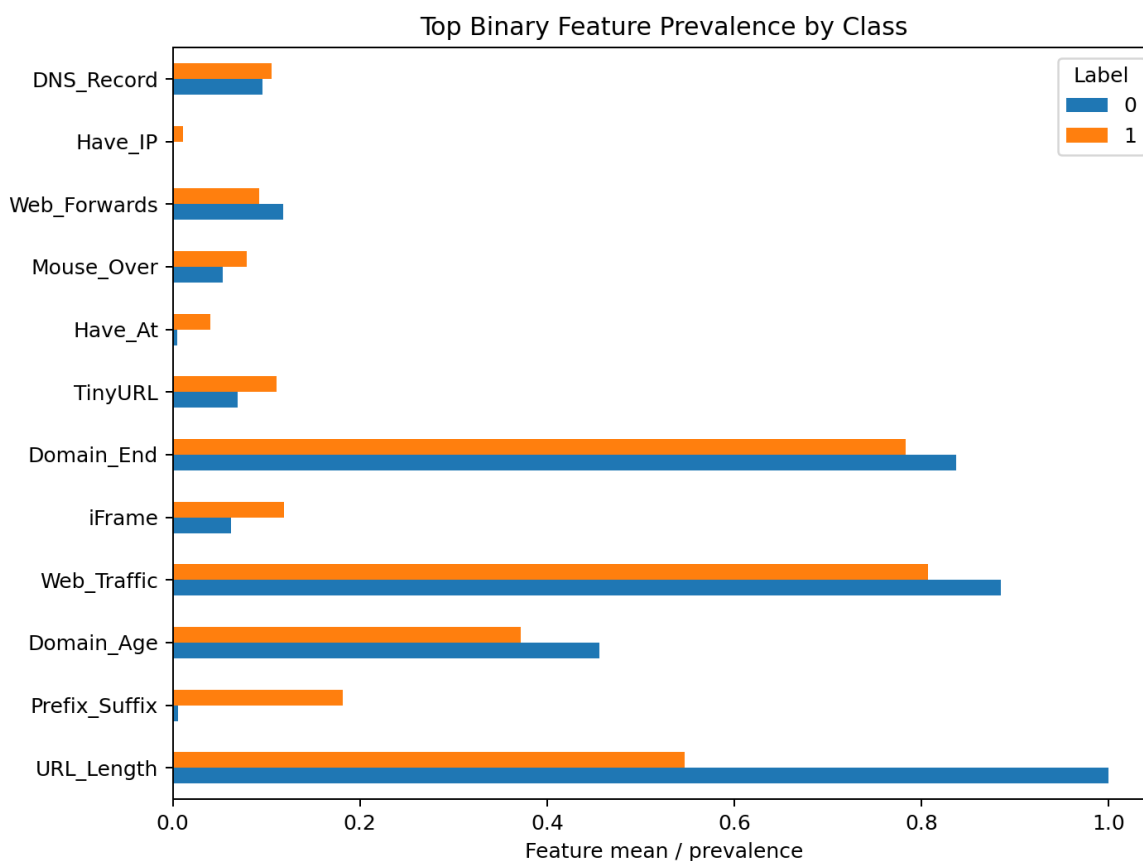


Figure 4. Largest class-conditional prevalence differences among binary features.

The following table reports random stratified split results. This split is closest to the original methodology and therefore useful for reproduction.

Random stratified 80/20 split results

Model/ Item	Accuracy	Precision	Recall	F1	F2	MCC	ROC_AU C	FP	FN
Random Forest	0.9590	0.9925	0.9250	0.9576	0.9378	0.9201	0.9948	7.0000	75.0000
Gradient Boosting	0.9225	0.9658	0.8760	0.9187	0.8926	0.8487	0.9725	31.0000	124.0000
Decision Tree	0.9195	0.9762	0.8600	0.9144	0.8810	0.8450	0.9607	21.0000	140.0000
Logistic Regression	0.8840	0.9257	0.8350	0.8780	0.8517	0.7717	0.9491	67.0000	165.0000

The grouped split by domain is a stricter robustness check. It prevents the same domain group from appearing in both training and test sets.

Grouped split by domain results

Model/ Item	Accuracy	Precision	Recall	F1	F2	MCC	ROC_AU C	FP	FN
Gradient Boosting	0.8708	0.8727	0.8784	0.8755	0.8772	0.7411	0.9391	119.0000	113.0000
Decision Tree	0.8685	0.8779	0.8665	0.8722	0.8688	0.7369	0.9115	112.0000	124.0000
Random Forest	0.8033	0.7531	0.9225	0.8292	0.8828	0.6203	0.9222	281.0000	72.0000

Logistic Regression	0.8178	0.8078	0.8504	0.8285	0.8415	0.6355	0.9245	188.0000	139.0000
---------------------	--------	--------	--------	--------	--------	--------	--------	----------	----------

The grouped split lowers the best F1 from about 0.958 to about 0.876. This does not invalidate the project, but it shows that the random split is optimistic and should not be the only evidence.

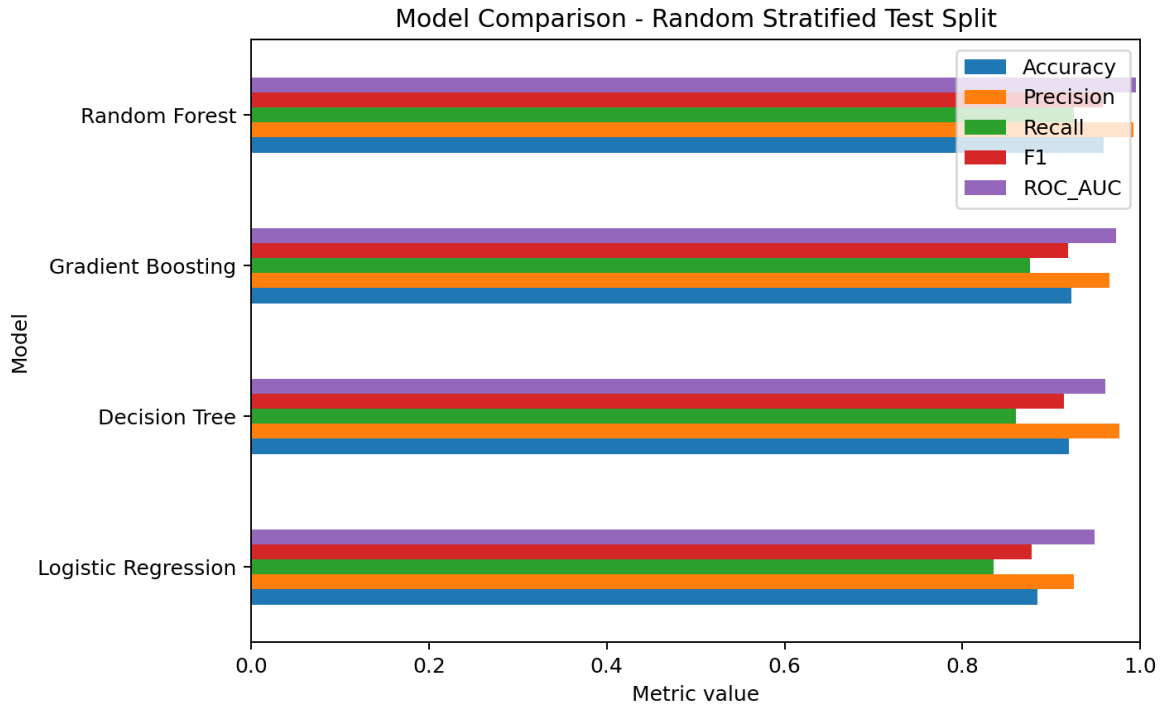


Figure 5. Model comparison on the random stratified split.

6. Evaluation Metrics and Error Analysis

For binary phishing detection, let TP be phishing URLs correctly detected as phishing, TN be legitimate URLs correctly accepted, FP be legitimate URLs incorrectly flagged as phishing, and FN be phishing URLs incorrectly accepted as legitimate. $\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}$. $\text{Precision} = \frac{TP}{TP+FP}$. $\text{Recall} = \frac{TP}{TP+FN}$. $\text{F1} = 2 * \text{Precision} * \text{Recall} / (\text{Precision} + \text{Recall})$. F2 gives recall more weight. MCC uses all four confusion-matrix cells and is useful when class imbalance matters. ROC-AUC measures ranking quality across decision thresholds.

In the cybersecurity context, false negatives are especially dangerous because users remain exposed to phishing pages and credential theft. False positives are also costly because they block legitimate work and create alert fatigue. Therefore, a deployment should choose a threshold based on operational risk, not simply use 0.5 by default.

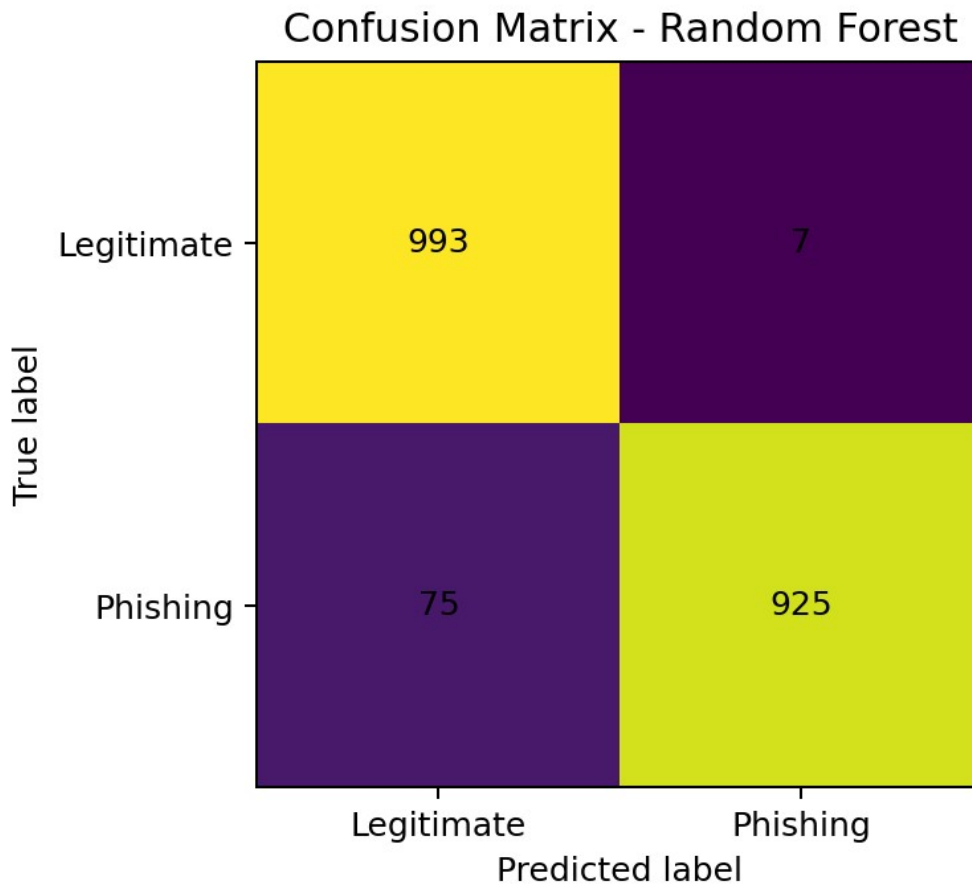


Figure 6. Confusion matrix for the best random-split model.

Threshold trade-off for the best random-split model

Model/Item	precision	recall	f1	f2	FP	FN
0.3	0.9709	0.9690	0.9700	0.9694	29.0000	31.0000
0.35	0.9786	0.9600	0.9692	0.9637	21.0000	40.0000
0.4	0.9835	0.9550	0.9691	0.9606	16.0000	45.0000
0.45	0.9884	0.9400	0.9636	0.9493	11.0000	60.0000
0.5	0.9925	0.9250	0.9576	0.9378	7.0000	75.0000
0.6	0.9989	0.9020	0.9480	0.9198	1.0000	98.0000
0.7	1.0000	0.8720	0.9316	0.8949	0.0000	128.0000

At threshold 0.50, the Random Forest model produced 7 false positives and 75 false negatives on the random test set. Lowering the threshold to 0.35 increased recall from 0.925 to 0.960, reducing false negatives from 75 to 40, but false positives increased from 7 to 21. In a high-risk anti-phishing product, this lower threshold may be preferable; in a user-facing browser blacklist, the extra false positives may be too disruptive.

7. Conclusions and Recommendations

- The source is a strong educational baseline for phishing detection with machine learning.
- The extracted features are meaningful: URL length, domain length, URL depth, prefix/suffix, and domain structure were among the strongest signals.
- The author's high-level conclusion is directionally supported: machine learning can detect phishing patterns in the provided dataset.

- The evidence is not sufficient for production deployment because of duplicates, no timestamps, artificial class balance, and missing temporal validation.
- The main future improvement is to evaluate on a time-based split with newly collected URLs and realistic phishing prevalence.
- Additional useful features could include WHOIS age, certificate metadata, DNS TTL, lexical entropy, brand impersonation scores, page text embeddings, screenshot similarity, and threat-intelligence reputation.

Final recommendation: use this project on similar problems as a baseline and teaching example, not as a final operational phishing detector. Before deployment, retrain and validate on fresh data, document the feature-extraction pipeline, calibrate probabilities, select thresholds according to security cost, and continuously monitor drift.

Appendix A. Course Lecture Alignment

The uploaded lecture pack was used to shape the methodology. EDA lectures motivated checking missing values, duplicates, single-value features, class prevalence, outliers, crosstabs, and feature distributions. The categorical-data and statistics lectures motivated Spearman correlation for ordinal/binary indicators and redundancy checks. The pipeline lecture motivated a reproducible sequence from loading to EDA, preprocessing, modeling, and evaluation. The goodness-of-fit and ROC-AUC lectures motivated precision, recall, F1/F2, MCC, ROC-AUC, and confusion-matrix discussion. The explainability lecture motivated feature-importance analysis. The social-engineering lecture motivated the emphasis on phishing risk and the false-positive/false-negative trade-off.

Appendix B. References

- Selected source: <https://github.com/shreyagopal/Phishing-Website-Detection-by-Machine-Learning-Techniques>
- Original data file: <https://github.com/shreyagopal/Phishing-Website-Detection-by-Machine-Learning-Techniques/blob/master/DataFiles/5.urldata.csv>
- PhishTank developer information: https://www.phishtank.com/developer_info.php
- University of New Brunswick URL dataset: <https://www.unb.ca/cic/datasets/url-2016.html>
- Course lecture pack supplied by the student: EDA, statistics, categorical data, pipeline, goodness-of-fit/metrics, explainability, and social engineering lectures.